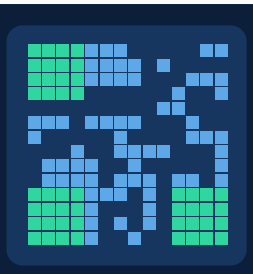
QRaksha

Technical architecture & judge-prep guide

"A UPI app can tell you who receives the money.
QRaksha tells you whether the QR in front of you was supposed to be there."



1 The problem — in plain words

You walk into a small shop, scan the QR on the counter, your UPI app shows a name, you pay. What you cannot see: someone pasted **their own QR over the shopkeeper's**. The money lands in the scammer's account, the shopkeeper never gets paid, and UPI payments are near-instant and very hard to reverse.

> **THE GAP** UPI apps verify **who will receive the money** once a QR is scanned. They cannot verify whether the **physical sticker** in front of you is genuine, cloned, or moved. That gap is what QRaksha closes — **before** the customer pays.

Attack	What the scammer does	Why UPI alone misses it
QR replacement	Pastes a fake QR over the genuine one	The fake is a perfectly valid UPI QR — it just pays the wrong person
QR cloning	Prints a copy of a genuine merchant QR and places it elsewhere	Payload is valid and belongs to a real merchant
QR relocation	Moves a genuine registered QR to the wrong physical place	Every standard check passes — the QR is real, only its *context* is wrong
Physical tampering	Overlays or alters the payment surface	Nothing in the payload changes

2 The solution in 30 seconds

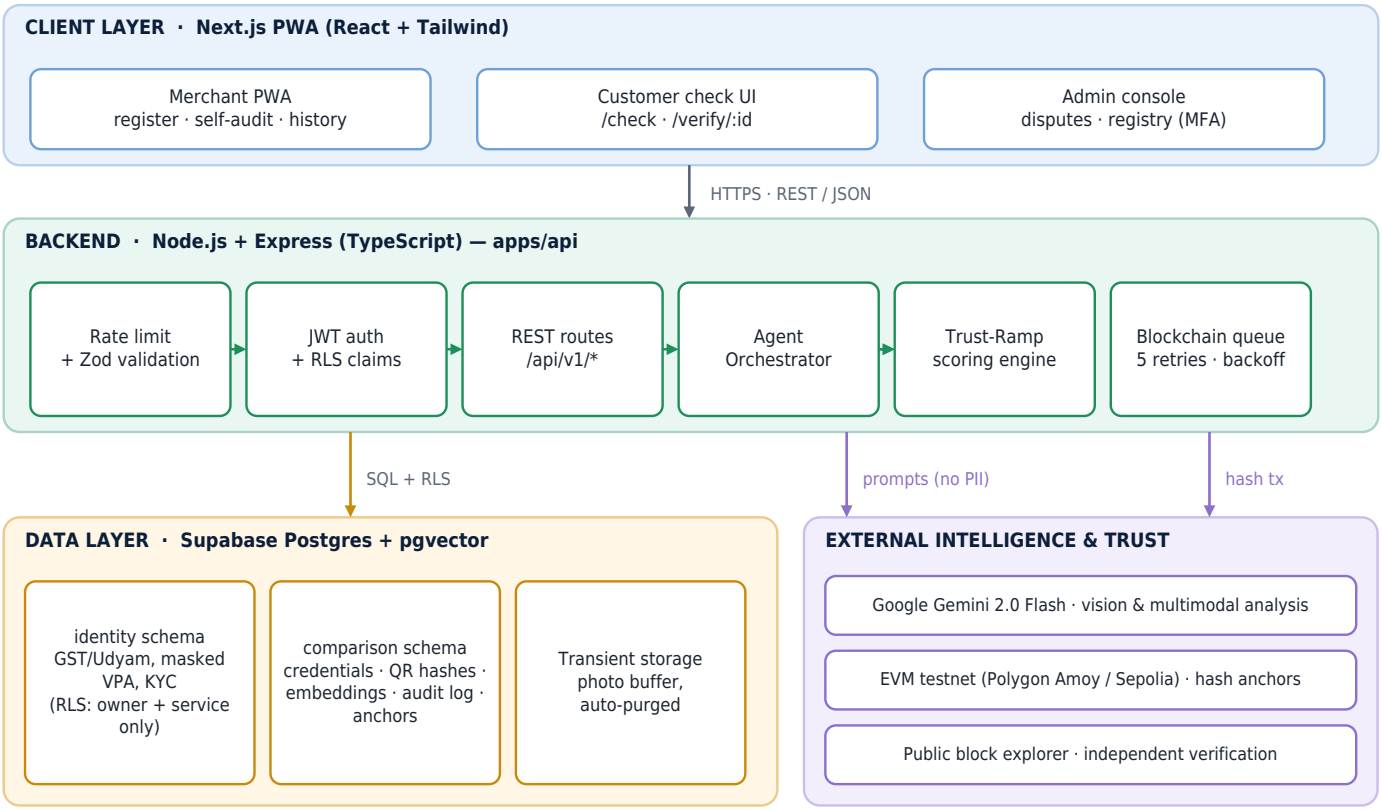
QRaksha is a **physical-trust layer for UPI QR codes**. Two sides, one idea: a merchant proves once — and keeps proving — that *this exact QR belongs at this exact place*; a customer can check that claim in seconds before paying.

Merchant side (register once, re-audit periodically)	Customer side (optional, before paying)
1. Identity check — GST / Udyam or penny-drop **2. Reference photo of the installed QR + GPS location → stored as a vector, photo discarded** **3. Proof of possession (₹1 micro-deposit, simulated in the MVP)** **4. Credential issued; hash anchored on a public blockchain** **5. Periodic self-audits (photo + QR + GPS) keep the credential fresh**	**1.** Scan the shop's QR with QRaksha (PWA, no install) before using GPay / PhonePe **2.** Agents compare payload, location, visuals and scan history **3.** Verdict in seconds: ● **VERIFIED** ● **UNVERIFIED / RECENTLY REGISTERED** ● **WARNING** **4.** Anyone can verify the credential on a public block explorer

Key distinction: existing UPI security answers "who gets my money?". QRaksha answers "does this physical QR actually belong to this merchant, here, now?" — including the case of a **genuine** merchant QR moved to the wrong place, which sails through every normal UPI check.

3 System architecture

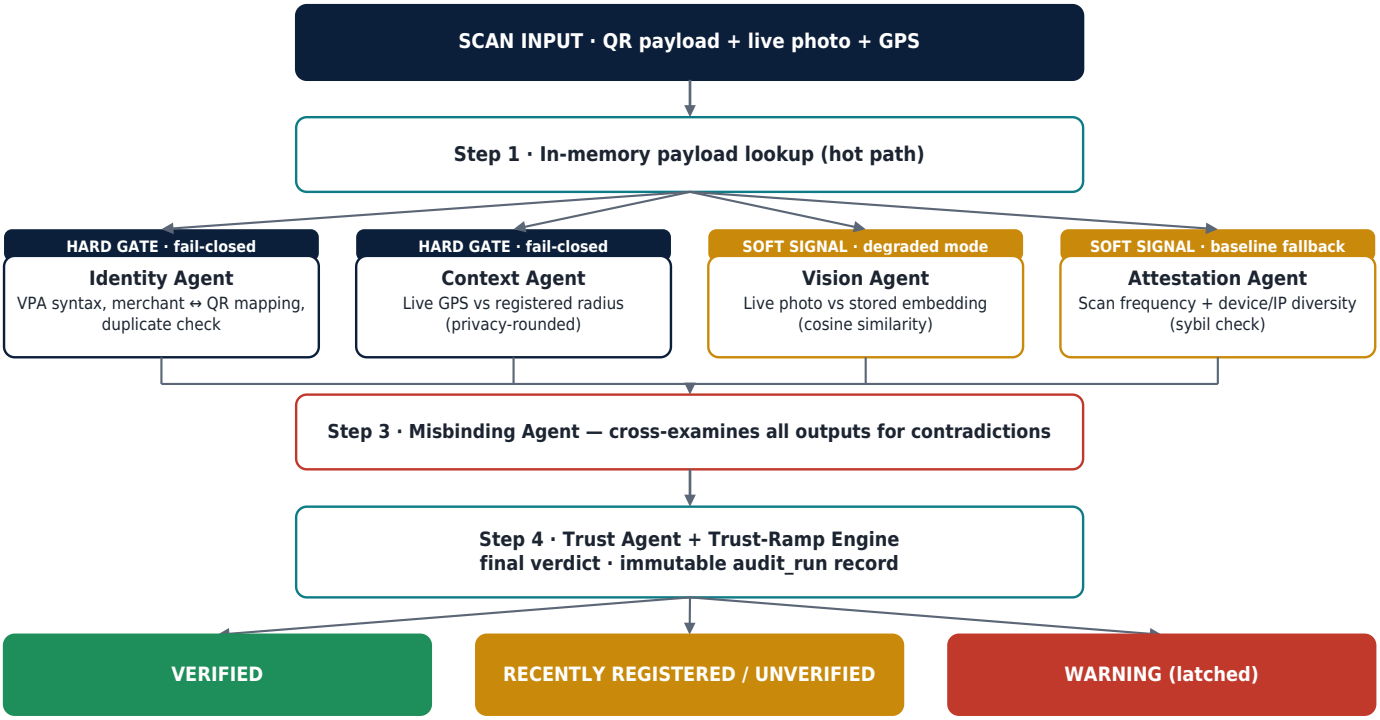
Four layers. The client captures; the API orchestrates and decides; Supabase stores identity and operational data in **separate, isolated schemas**; external services supply vision intelligence (Gemini) and public tamper-evidence (EVM testnet).



Layer	What it does	Design choice worth saying out loud
Client (Next.js PWA)	Merchant registration/audit, customer /check and /verify/:id, admin dispute console	PWA = camera + GPS in the browser, no app-store install for merchants or customers
Backend (Express + TS)	Rate limiting, Zod validation, JWT + RLS claims, REST routes, agent orchestration, trust scoring, async blockchain queue	Anchoring is **non-blocking** — the audit never waits on the chain
Data (Supabase Postgres + pgvector)	identity schema (PII) and comparison schema (credentials, QR hashes, embeddings, append-only audit log, anchors, disputes)	PII physically separated from operational data — a breach of one is not a breach of both
External	Gemini 2.0 Flash for multimodal analysis; Polygon Amoy / Sepolia for hash anchoring; block explorer for public verification	Gemini only receives an allow-listed set of fields (see §6)

4 The verification pipeline — how a verdict is made

A scan runs through a staged pipeline: a fast deterministic lookup, four specialist agents in parallel, a cross-checking Misbinding agent, then the Trust agent. The key design idea: **deterministic checks are hard gates; probabilistic AI is a soft signal.** The system never lets a vision model alone approve a payment.

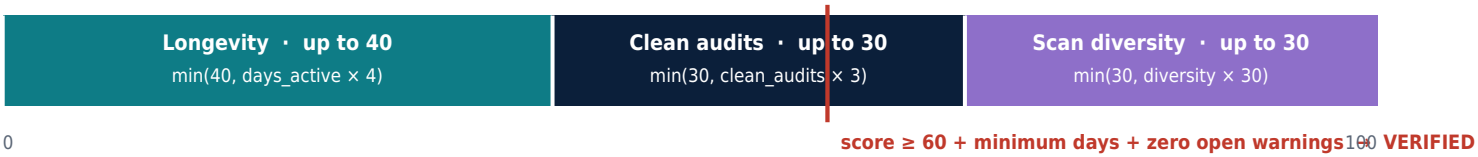


Agent	What it checks	If it fails
Identity	UPI string structure, merchant VPA mapping, registration state in registered_qr	**Hard:** pipeline aborts with 503 rather than risk a false approval
Context	Live GPS vs bound radius (bound_radius_m); GPS is privacy-rounded	**Hard:** relocation → instant WARNING; outage aborts pipeline
Vision	Live photo → vector → cosine similarity vs stored reference_embedding	**Soft:** pipeline continues, flagged degraded_check: true
Attestation	Scan frequency + source IP/device diversity to spot synthetic / sybil scans	**Soft:** falls back to a baseline weight
Misbinding	Cross-checks all outputs — e.g. valid payload but contradictory photo or location	Reasoning step feeding the Trust agent
Trust	Merges scores into a verdict + trust-ramp transition; writes an immutable audit_run	Emits the final AuditVerdict

TEAM NOTE The diagram shows four specialist agents + Misbinding + Trust. Some project docs say “five-agent pipeline” — agree on one name (e.g. “four specialists, one cross-checker, one decider”) before the demo so judges don't count and catch you.

5 Trust-ramp: trust is earned, never granted on day one

A newly registered sticker starts at **RECENTLY REGISTERED**. A scammer who registers today cannot look “verified” today — trust needs **time, clean audits and diverse scans**.



Verdict	Rule
VERIFIED	trust_score ≥ 60 **and** days_active ≥ 3 (GST-verified) or ≥ 7 (baseline) **and** no unresolved warnings
RECENTLY REGISTERED / UNVERIFIED	trust_score < 60 and no unresolved warnings
WARNING (latch)	Triggered instantly on payload mismatch, location outside radius, or visual misbinding. **Never auto-clears** on a later clean scan — needs manual admin dispute resolution

Why the latch matters: an attacker cannot “wash out” a red flag by following a bad scan with a good one. A human has to resolve it, with mandatory notes on the record.

6 Security & privacy by design

Schema isolation. Sensitive PII (gst_udyam_number, bank_vpa_masked, kyc_address) lives only in the identity schema — readable by the owning merchant and the backend service role. Even platform admins cannot select raw PII rows. The comparison schema holds only foreign keys, embeddings and hashes.

Zero-persistence photos. A photo is received only to extract a feature vector, then immediately purged. QRaksha never keeps merchants' raw photos.

AI data boundary (enforced in ai-guard.ts):

Allowed to reach Gemini	Strictly forbidden from Gemini
Decoded QR payload string & hash	Merchant GST / Udyam numbers
Live audit image — request lifecycle only	Bank account numbers & raw VPAs
GPS rounded to ~100 m precision	Admin credentials & internal keys
Numeric embedding vectors	Other merchants' cross-tenant data

7 Blockchain anchoring — what it is, and what it isn't

Each credential's fingerprint is published on a public EVM testnet as a **zero-value transaction whose calldata is a SHA-256 hash** of credential_id + merchant_id + qr_payload_hash + created_at. No smart contract, no gas for merchants (a custodial relayer pays), no PII on-chain.

- **Does:** give anyone — customer, bank, auditor — an independent, tamper-evident proof that this credential existed, unchanged, at that time. No need to trust QRaksha's database.
- **Does not:** detect fraud. Detection is the agent pipeline; the chain is the trust anchor underneath it.
- **How:** async in-process queue with exponential backoff (5 retries); resulting tx_hash stored in blockchain_anchor; explorer link returned by GET /verify/:credentialId.

8 Tech stack & API at a glance

Component	Technology	Role
Frontend	Next.js PWA, React, Tailwind	Camera + GPS capture, merchant & customer UI

Component	Technology	Role
Backend	Node.js + Express (TypeScript)	REST API, orchestration, anchoring
Database	Supabase Postgres + pgvector	Dual schema with RLS; vector similarity
AI engine	Google Gemini 2.0 Flash	Structured-JSON multimodal analysis
Blockchain	Polygon Amoy / Sepolia, ethers.js	Calldata hash anchoring
Validation / ops	Zod (shared package), Sentry, JSON logs	End-to-end schemas; correlation IDs across agent verdicts

Endpoint	Auth	Purpose
POST /merchants/register/identity	Public	GST/Udyam or penny-drop identity verification
POST /merchants/register/reference	Merchant JWT	Reference photo + GPS → vector stored, photo discarded
POST /merchants/register/proof-of-possession	Merchant JWT	₹1 micro-deposit verification (simulated)
POST /audit/run	Merchant JWT	Full multi-agent audit (photo + QR + GPS)
GET /audit/history	Merchant JWT	Paginated audit verdicts
POST /check	Public	Lightweight customer verification before paying
GET /verify/:credentialId	Public	Status, trust score, explorer URL
POST /disputes · PATCH /admin/disputes/:id	Merchant JWT · Admin (MFA)	Raise / resolve disputes with mandatory notes

9 The two-demo story

- **Demo 1 — Replacement.** A registered merchant QR is swapped for a fake. QRaksha flags it: the payload doesn't match the registered credential.
- **Demo 2 — Relocation (“the killer demo”).** The *real* registered QR is physically moved elsewhere. Payload is valid, UPI would approve it — QRaksha returns **WARNING**: right QR, wrong place.

Close on Demo 2 — it is the one thing a normal UPI app structurally cannot do.

10 Hard-hitting questions judges can ask — and how to answer

Answers are short on purpose: say the first two sentences, then stop and let them dig. **Prep** notes flag things to confirm in your code or demo before you claim them.

Q1. UPI apps already show the merchant's name, and shops have soundboxes. Why do we need this?

The app shows the name tied to whichever QR you scanned — if the scammer's QR is on the counter, the name is the scammer's, and most customers don't know what name to expect. A soundbox only tells the *real* shopkeeper that money didn't arrive — after you've already paid, and UPI is near-instant and hard to reverse. QRaksha works **before** payment and asks a different question: not “who gets the money?” but “was this QR supposed to be here?”

Q2. Why would a customer open a separate app before paying?

They won't for every purchase — and don't need to. The high-risk moments are unfamiliar shops, street vendors and large payments. It's a PWA, so there's nothing to install. Adoption is merchant-driven: shopkeepers protect their own revenue and display the verify link. Long term, the check is an API that UPI/PSP apps can embed so it becomes invisible.

> **PREP** Have one clear go-to-market sentence ready: who adopts first (merchants in high-fraud settings) and why.

Q3. A scammer can register honestly under his own GST, then upload a photo of someone else's shop. How do you stop that?

Layers, not a single wall: (1) GST/Udyam + proof-of-possession ties the credential to a real, traceable identity; (2) the trust ramp means nothing is VERIFIED on day one — it needs days, clean audits and scan diversity; (3) GPS is bound at registration and re-checked on every audit; (4) genuine shopkeepers can raise disputes against hijacked credentials. We won't claim zero fraud — we raise the cost, add traceability and delay trust. A natural next step is cross-checking registered GPS against the GST-registered address.

Q4. Shops get renovated, lighting changes, and one shop can have several QRs. Doesn't photo matching break?

We don't do exact image matching — Vision compares embeddings with cosine similarity, which tolerates lighting and decor change, and it is a **soft signal**: it can degrade confidence but is never the sole reason for a red verdict. Hard checks (payload identity, location) don't depend on the photo. Each QR payload is its own registered_qr record, and periodic self-audits keep references current.

> **PREP** Confirm the “refresh reference photo” flow exists in the UI before saying merchants can update it.

Q5. Why blockchain? Isn't a normal database enough?

A database is enough for *detection*; the chain solves “why should anyone trust your database?”. Anchoring a SHA-256 of the credential on a public chain lets any customer, bank or auditor verify independently that it existed unchanged at that time. Only a hash goes on-chain — no PII, no photos. Honest framing: it adds tamper-evidence, it does not detect fraud, and today it runs on a testnet.

Q6. Isn't this just a malicious-QR / phishing-link scanner?

No. Those tools check what a QR *points to*. In this attack the QR is a perfectly legitimate UPI payload — the fraud is its **physical placement**. QRaksha checks payload, location, appearance and scan behaviour together.

Q7. The AI is probabilistic. What about false alarms and misses?

AI never decides alone. Identity and Context are hard, deterministic gates against registered data; Vision and Attestation are soft signals with a degraded mode. Every audit is stored in an append-only audit_run log, and a red verdict latches until a human admin resolves it. The cost of a false alarm — a genuine merchant flagged — is exactly why the dispute workflow exists.

Q8. What if Gemini or a service goes down?

The system fails **closed** where it matters: if Identity or Context can't run, the API returns 503 instead of approving. If only Vision fails, the audit continues and is clearly marked degraded_check: true. The principle: an outage can make us less helpful, never less safe.

Q9. A fake QR is pasted exactly over the real one, same location. Yellow or red?

The fake's payload doesn't match the merchant's registered payload, so Identity can't bind it. The customer's GPS sits inside a registered merchant's boundary, yet the scanned QR belongs to nobody there — that contradiction is exactly what the Misbinding agent exists to catch.

> **PREP** Run this exact scenario through orchestrator.ts and confirm it returns red (not just yellow “unverified”). It's Demo 1 — don't guess.

Q10. What if the scammer clones the genuine QR and puts it in another shop?

The payload is valid, so Identity passes — that's the point of the killer demo. The scan GPS falls outside the bound radius, so Context triggers an instant WARNING; Vision and Attestation add supporting evidence. The warning latches until an admin resolves it.

Q11. Can the trust score be gamed with fake scans?

The score has caps (40 longevity / 30 audits / 30 diversity) and VERIFIED additionally requires minimum elapsed days and zero open warnings — so fake scans can't buy time, and any red flag overrides the score. Honestly, the diversity signal is the most gameable part for an attacker with many devices; that's why it is only one of three inputs and always paired with time.

Q12. Is it fast and scalable? Does every customer scan call Gemini?

The customer path is the lightweight one: payload lookup first, in memory. The heavy multimodal audit is merchant-initiated and periodic. Blockchain anchoring is asynchronous with retries, so it never blocks a scan. Target for an audit is under 3 seconds.

> **PREP** Measure real end-to-end latency before quoting a number on stage.

Q13. Privacy: where does the data go? What about insider or key risk?

Gemini only receives an allow-listed set (payload hash, transient image, ~100 m rounded GPS, vectors). PII sits in a separate RLS-protected schema that even admins can't read raw. Photos are purged after vector extraction. Admin actions need MFA and mandatory resolution notes. The relayer key can only publish hashes — compromise can't rewrite on-chain history — though production would move it to managed KMS/HSM.

Q14. Who pays, and what's the business model?

Our hypothesis: the parties who absorb fraud loss and trust risk — merchants (subscription), and PSPs / acquiring banks (API per check). Customer-side checks stay free to drive adoption. These are hypotheses to validate with pilot merchants, not proven numbers.

Q15. What is actually built versus planned?

Be upfront: the ₹1 proof-of-possession is simulated; the chain is a testnet; the demo registry is a small set of fictional merchants. The pipeline, trust ramp, dual-schema design and anchoring flow are the working core. Judges reward honesty far more than a bluff that collapses in follow-up.

> **PREP** Decide in advance, as a team, exactly which items you will call “working”, “simulated” and “roadmap”.

11 Known limitations — say them before the judges do

- **Two-sided adoption.** A green verdict needs a registered merchant; unregistered shops can only be “unverified”.
- **Vision is probabilistic.** It supports decisions; it never makes them alone.
- **GPS can be spoofed** on rooted devices — one reason location is combined with photo, time and scan diversity.
- **Diversity signal** can be gamed by a determined attacker with many devices; time requirements limit the damage.
- **Custodial relayer** is a single operational point; production needs managed key custody.
- **MVP scope:** simulated ₹1 deposit, testnet anchoring, fictional merchant registry.

30-SECOND PITCH **1.** UPI proves who gets the money — not whether the sticker on the counter is real. **2.** QRaksha lets merchants prove that this exact QR belongs at this exact place, and lets customers check it in seconds, before paying. **3.** Deterministic checks are the gate, AI is the evidence, and a public blockchain anchor makes the trust tamper-evident. **Close:** “A UPI app can tell you who receives the money. QRaksha tells you whether the QR in front of you was supposed to be there.”